



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE



User Stories, Backlog e Priorização Formal

Da conversa informal ao artefato formal,
estruturando o que sua equipe já faz
intuitivamente



O que vamos falar hoje

Cinco habilidades concretas para aplicar ainda esta semana:

01

Escrever User Stories

Formato padrão com persona
identificada e benefício claro

02

Avaliar com INVEST

Checklist objetivo para medir a
qualidade de cada story

03

Critérios de Aceitação

Formato Gherkin: Dado / Quando /
Então

04

Priorizar com MoSCoW

Classificar e defender cada escolha com uma justificativa

05

Decompor o Backlog

Distinguir Epic, Feature, Story e Task — e saber quando
quebrar



Segunda prepara, Quarta estrutura, Quinta executa

1

Segunda-feira

Visão ágil, papéis e mindset — stories escritas como lição de casa

2

Hoje — Quarta

User stories, INVEST, Gherkin, hierarquia e priorização MoSCoW

3

Quinta-feira

Sessão com stakeholder real, backlog priorizado na mão

📌 O que você aprender hoje vai diretamente para a quinta.





**"A parte mais difícil da
construção de um
software é a definição
do que se deve
construir"**

-- Fred Brooks



Do Requisito à User Story

A mudança parece pequena, mas muda tudo: o foco sai do sistema e vai para a pessoa que usa o sistema.



O que são requisitos?

- O que um sistema deve **fazer** e **sob que restrições**
- "o que um sistema deve fazer": requisitos funcionais
 - suas funcionalidades
- "sob que restrições": requisitos não-funcionais
 - Desempenho, usabilidade, segurança, disponibilidade, etc



Compare os dois lados

Requisito Tradicional

"O sistema deve autenticar usuários com e-mail e senha."

- Foco no sistema
- Não há persona
- Não diz por que importa

User Story

"Como **usuário cadastrado**, quero **fazer login com e-mail e senha** para **acessar minha conta com segurança**."

- Foco no usuário
- Persona identificada
- Benefício explícito

"Requisitos tradicionais descrevem o sistema. User stories descrevem o que o usuário precisa fazer — e por que isso importa para ele."



O Formato Padrão da User Story

Como [persona]

Força você a pensar em **quem usa**. Nem todo usuário é igual — um gestor tem necessidades diferentes de um cliente final.

Quero [ação]

Descreve a **ação desejada**, não a solução técnica. Evita pré-decidir como o sistema vai funcionar.

Para [benefício]

Ancora a story no **valor real**. Impede "features pelo feature" — se não tem benefício claro, a story não deveria existir.



Workshop 1: Revisão em Duplas



Abra as stories que você trouxe.

Antes de aprender os critérios formais,
você vai praticar agora.



Três perguntas para revisar em dupla

Para cada story que você e seu par trouxeram, apliquem estas três perguntas.

Tempo: **10 min em duplas + 5 min de plenária.**

1 A persona está identificada?

"Usuário" genérico não conta. Quem especificamente usa essa funcionalidade?

2 O benefício está claro?

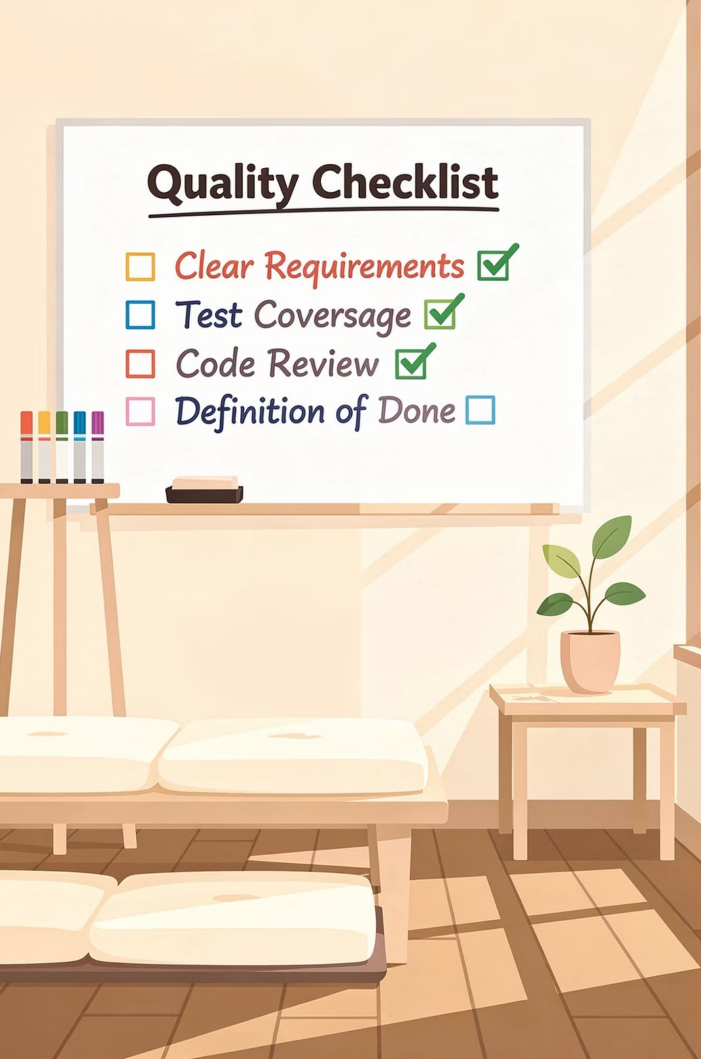
"Para ter uma melhor experiência" é vago demais. O benefício precisa ser específico e verificável.

3 Dá para testar?

Se você não consegue descrever como saber que a story foi entregue, ela precisa ser reescrita.

 **Normalize o imperfeito:** a primeira versão de toda story é sempre ruim. O workshop existe para melhorar — não para produzir stories perfeitas do zero.





INVEST: o checklist de qualidade das User Stories

Seis critérios para avaliar qualquer story de forma objetiva. Use como checklist antes de colocar uma story no backlog.



As 6 letras do INVEST



I — Independent

Pode ser desenvolvida sem depender de outra story?



N — Negotiable

É uma conversa aberta, não um contrato imutável.



V — Valuable

Entrega valor real para o usuário — não apenas para o dev.



E — Estimable

É possível estimar o esforço necessário para entregá-la.



S — Small

Cabe dentro de um sprint. Se não cabe, quebre.



T — Testable

Dá para saber se está pronta? Se não, reescreva.



Exemplos de stories que violam o INVEST

Reconheça os padrões, eles aparecem em todo backlog real.

Critério	Exemplo problemático	Por que viola?
I	"Quero fazer checkout — mas só depois que o carrinho estiver implementado."	Dependência explícita de outra story
V	"Como dev, quero refatorar o banco de dados para melhorar a performance."	Tarefa técnica, sem valor direto ao usuário
S	"Como gestor, quero gerenciar usuários, produtos, pedidos e relatórios."	São 4 stories disfarçadas de uma (Epic)
T	"Como usuário, quero que o sistema seja rápido."	"Rápido" não é verificável sem métrica



Critérios de Aceitação

Como você sabe quando uma user story está realmente pronta?

"Pronto" precisa ser um fato verificável, não uma opinião.

Given

- Givenmesconphian
- Thar sffeeregpe →

When

- Torromecscarikion
- Thar sffeeregpe →

Then

- Fairnmessonphion
- Thar sffeereppe



O Formato Gherkin: Dado / Quando / Então

A estrutura

Dado que [contexto / pré-condição]

Quando [ação do usuário]

Então [resultado esperado]

Cada cenário é testável. Pode virar um teste automatizado na quinta-feira.

Story de exemplo

"Como usuário, quero redefinir minha senha para recuperar o acesso à conta."

Critério 1 – Caminho feliz: Dado que está na tela de login

Quando clica em "Esqueci minha senha" e informa o e-mail

Então recebe e-mail com link válido por 1 hora

Critério 2 – Caso de falha: Dado que o link já expirou

Quando tenta acessá-lo

Então vê mensagem de erro e pode solicitar novo link



Três insights sobre critérios de aceitação

Casos de falha são tão importantes quanto sucessos

Um critério de aceitação que só cobre o caminho feliz não protege o usuário. Sempre pense: "o que acontece quando algo dá errado?"

Requisitos não funcionais aparecem aqui

"Mínimo 8 caracteres" é um requisito de segurança. Critérios de aceitação são onde eles se tornam verificáveis e rastreáveis.

Cada critério pode virar um teste automatizado

Na quinta-feira, esses critérios orientam diretamente o que será validado com o stakeholder — e mais tarde, os testes do sistema.

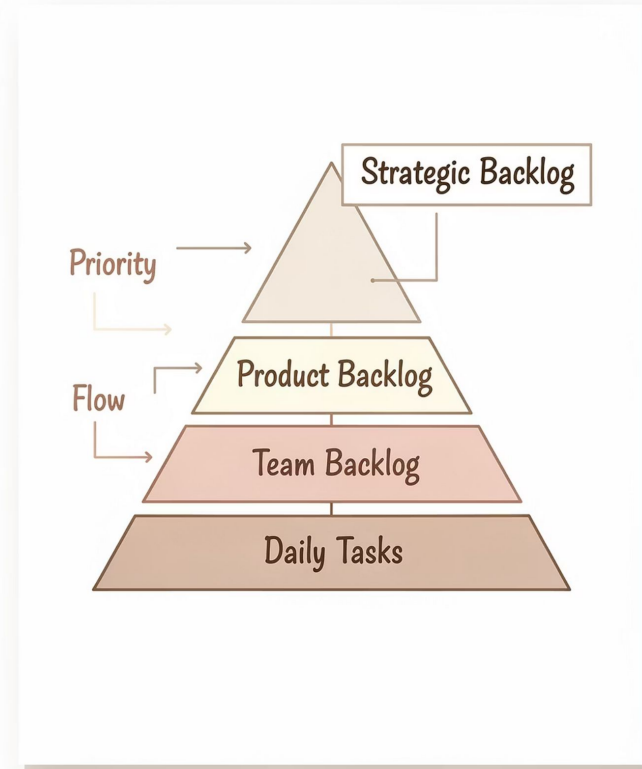
📝 **Agora pratique:** escreva um critério de aceitação para uma das stories que você trouxe. Tempo: 5 minutos.



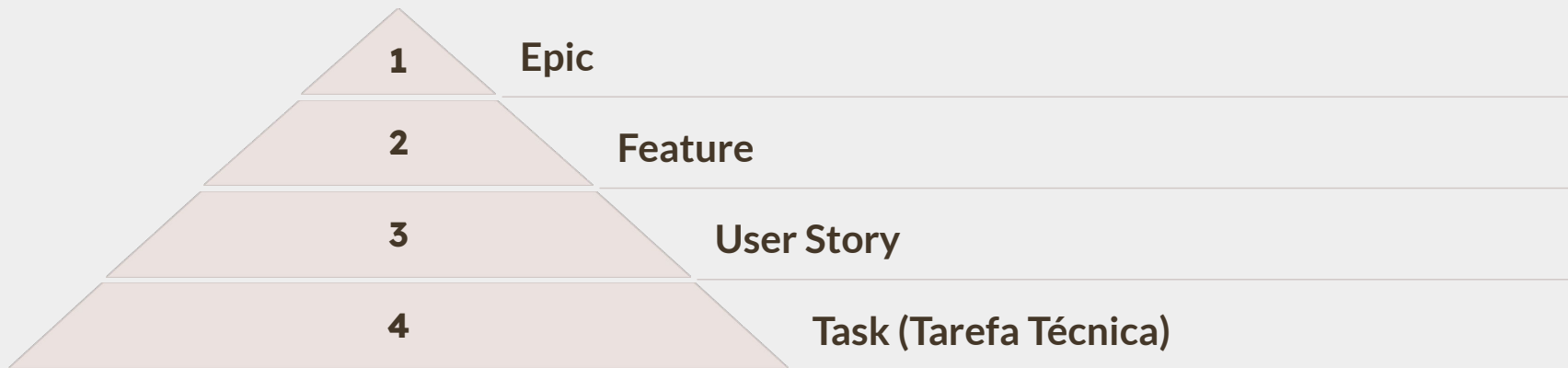
Hierarquia do Backlog e MoSCoW

Antes de priorizar, você precisa saber o que está priorizando.

Nem toda "story" é uma story, e confundir os níveis gera backlog inchado e sprint impossível.



Epic → Feature → Story → Task



Exemplo concreto — Autenticação:

Epic: Autenticação de usuários

Feature: Login com e-mail e senha

Story: "Como usuário cadastrado, quero fazer login para acessar minha conta."

Task: Criar rota POST /auth/login no backend

- ❑ **Regra prática:** se a story não cabe num sprint de 2 semanas, provavelmente é uma Feature ou Epic.
Quebre agora



MoSCoW: priorizando o que importa

Quatro categorias para forçar uma conversa honesta sobre escopo — com o time e com o stakeholder.

Must Have

Sem isso, o produto não funciona ou perde todo o valor.
Não negocie.

Should Have

Alto valor, mas não impede o lançamento. Pode entrar no Sprint 2.

Could Have

Desejável se houver tempo e esforço. Descartável sem grande impacto.

Won't Have (now)

Reconhecido e explicitamente adiado. Não é "nunca" — é "não agora".



A Armadilha dos 80% Must Have

Quando o cliente classifica quase tudo como Must Have, o problema está na conversa, não nas funcionalidades. Use estas perguntas para destravar:

"Se você pudesse lançar apenas **3 funcionalidades** amanhã, quais seriam?"

"O que acontece com seu **negócio** se essa funcionalidade **não existir no lançamento?**"

"Quem ficaria **prejudicado** se adiássemos isso para a **segunda versão?**"

📌 **Won't Have não é Never Have.** Comunicar ao stakeholder o que ficou fora é tão importante quanto comunicar o que entrou.



Workshop 2: Priorize com MoSCoW

A instrução

Abra o backlog do seu grupo. Para cada story, classifique em **M**, **S**,

C ou **W**. Regras obrigatórias:

No máximo **40%** das stories podem ser Must Have

Pelo menos **20%** devem ser Won't Have (now)

Toda classificação precisa de **uma justificativa em uma frase**

Por que essas regras?

Os limites de 40%/20% são artificiais — mas forçam o time a **negociar explicitamente** em vez de classificar tudo como urgente. Esse é exatamente o objetivo.

Após o workshop: uma equipe compartilha uma decisão difícil de priorização e como justificou.

Debate aberto por 2–3 min.





Desafios Observados na Gestão Ágil de Requisitos



Equilíbrio Agilidade-Documentação

Encontrar o ponto ideal entre manter a agilidade e ter documentação suficiente para referência futura.



Alinhamento de Stakeholders

Manter todos os envolvidos alinhados e engajados ao longo do processo iterativo de desenvolvimento.



Controle de Escopo

Gerenciar mudanças frequentes sem perder o foco nos objetivos principais do projeto.



Garantia de Qualidade

Assegurar que os requisitos sejam claros, testáveis e atendam às necessidades reais dos usuários.



Lições Aprendidas

1

Comunicação é Chave

A comunicação frequente e clara entre todos os membros da equipe e stakeholders é fundamental para o sucesso do projeto.

3

Feedback Contínuo

Incorporar feedback regularmente ajuda a manter o projeto alinhado com as expectativas e necessidades dos usuários.

2

Flexibilidade com Propósito

Ser ágil não significa ausência de planejamento, mas sim estar preparado para adaptar-se rapidamente às mudanças.

4

Equilíbrio é Essencial

Encontrar o equilíbrio certo entre documentação, desenvolvimento e colaboração é crucial para o sucesso do projeto ágil.



Você saiu desta aula com o essencial

Stories escritas

Formato padrão com persona, ação e benefício real

INVEST aplicado

Checklist para avaliar qualidade antes de qualquer sprint

Gherkin dominado

Critérios de aceitação verificáveis, com casos de falha

Backlog priorizado

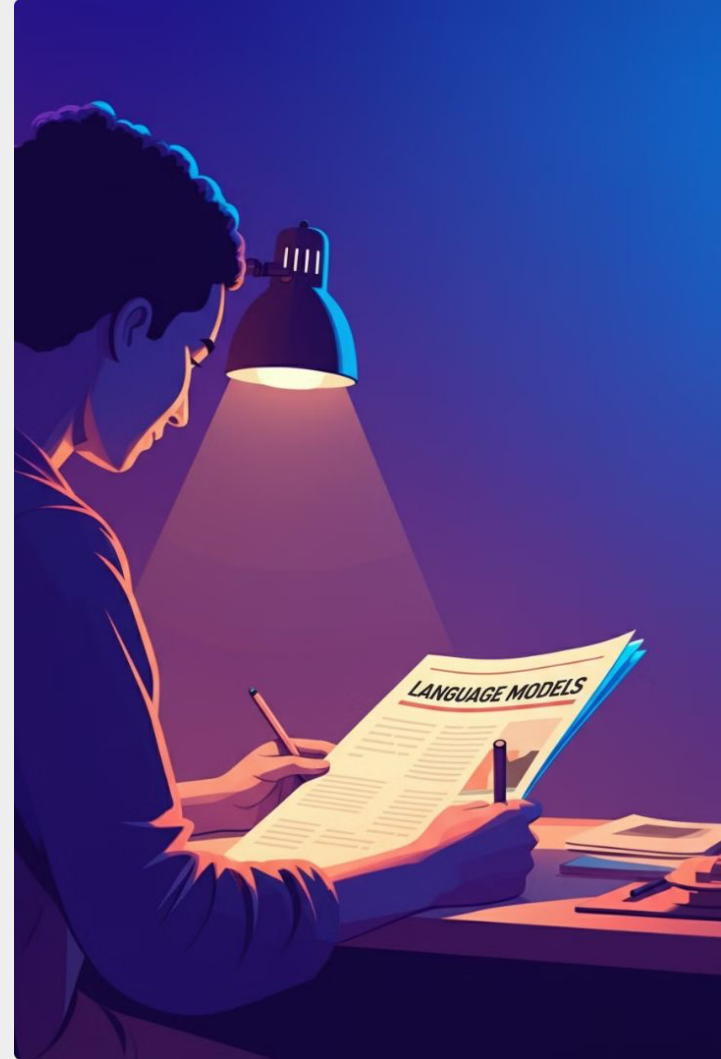
MoSCoW aplicado com justificativas

A primeira versão do backlog nunca está perfeita. O que importa é que está estruturado o suficiente para a conversa com o stakeholder começar bem.



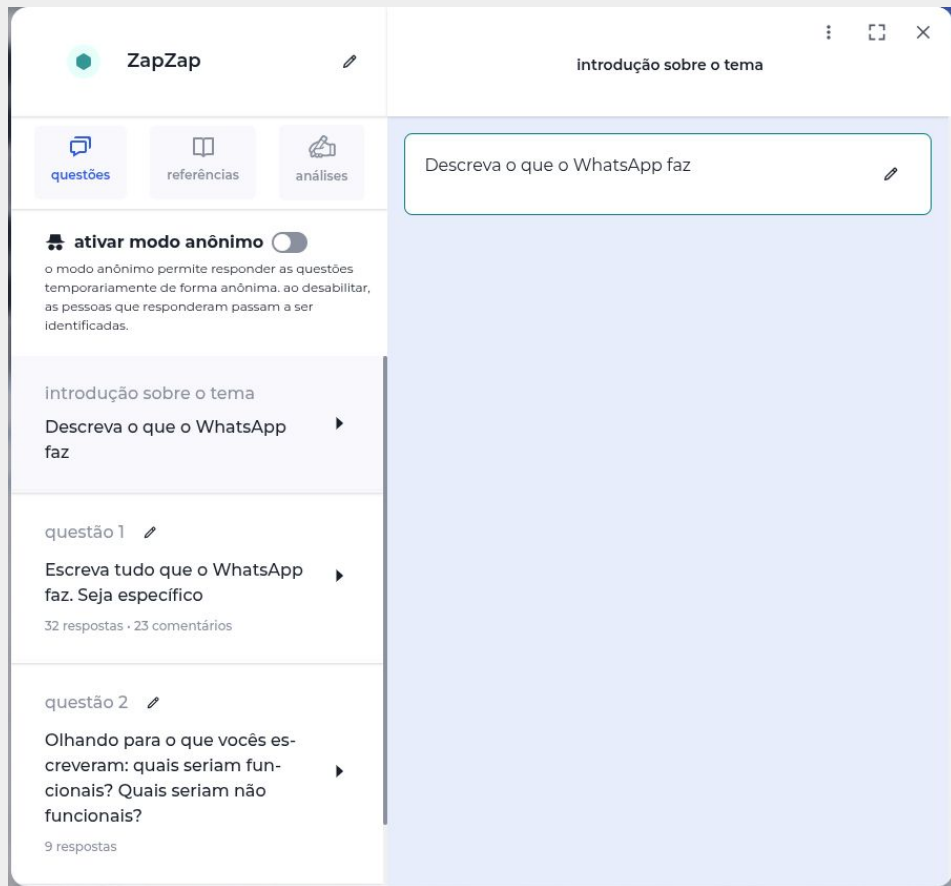
Próximos passos

- Eng. Software em Dimensões
 - Cap. 6 (seções 6.1–6.2): Requisitos funcionais e não funcionais, elicitação
 - Cap. 7 (seções 7.1.2–7.2.4): User Stories, backlog, MoSCoW, WSJF
- Eng. Software Moderna
 - Cap. 3 – Requisitos Histórias de Usuário, MVP, priorização



Workshop WhatsApp

1. Escreva tudo que o WhatsApp faz. Seja específico. [Segunda, Questão 01]
2. Transforme a sua descrição em pelo menos três users stories. Utilize o formato: "*Como [persona], quero [funcionalidade] para [benefício].*" [Segunda, comentário na sua resposta]
3. Responda a Questão 02
4. Para cada story que você descreveu, apliquem estas três perguntas.
 - a. A persona está identificada?
 - b. O benefício está claro?
 - c. Dá para testar?
5. Corrija suas user stories



contacts

vcg@cin.ufpe.br
@vinicius3w
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

User Stories, Backlog e
Priorização Formal